

Reading LiDAR Data

I. Experiment Objective

Learn how to receive the ranging data of the T-mini Plus LiDAR through the UART serial port on the ESP32-S3 platform. Master LiDAR data protocol parsing, ring buffer management, and multi-angle distance reading methods, laying the foundation for subsequent mapping, navigation, and obstacle avoidance functions.

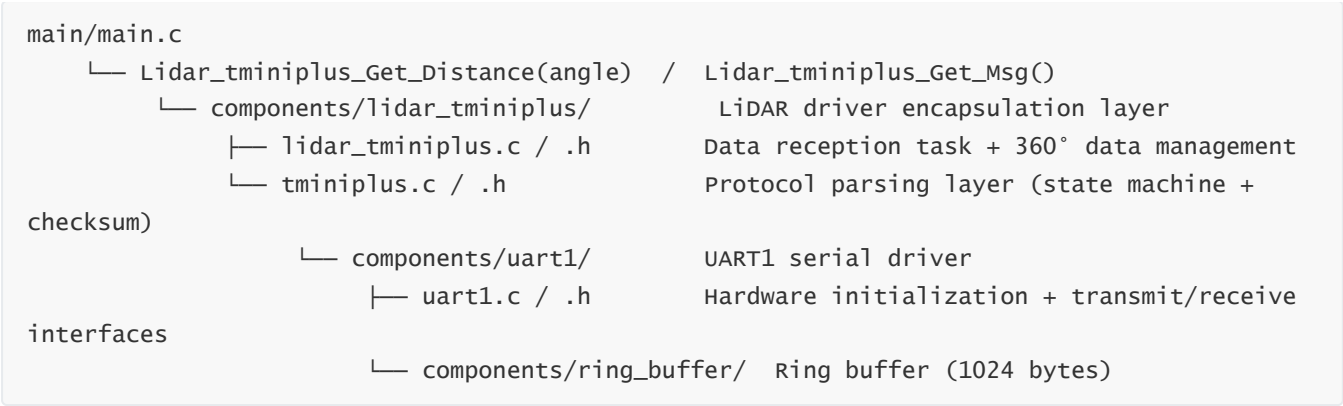
II. Experiment Principle

1. Keywords

Keyword	Explanation
LiDAR	A sensor that measures the distance to surrounding objects by emitting laser beams and measuring the time difference of the reflected light (TOF). The T-mini Plus is a 360° single-line LiDAR with a maximum ranging distance of 8m and a resolution of 1°, outputting multiple sets of distance data per second.
T-mini Plus	A small, low-cost 360° LiDAR that uses the TOF ranging principle and outputs ranging data through a UART serial port (baud rate 230400). There are 360 sampling points per revolution, and the data includes angle, distance (mm), and signal strength.
UART serial communication	Universal Asynchronous Receiver/Transmitter. This experiment uses UART1 (TX:GPIO17, RX:GPIO18) with a baud rate of 230400 in 8N1 mode. The LiDAR data is continuously transmitted to the ESP32 through the RX pin.
Ring buffer	A fixed-size (1024 bytes) first-in-first-out circular queue used as a temporary cache for UART data reception. It solves the mismatch between the serial data reception rate and the CPU processing rate, avoiding data loss.
Protocol parsing state machine	Parses the LiDAR data packets byte by byte through a state machine: detect the packet header (0x55AA) → parse the packet type/length → receive the sampling data → checksum verification. After verification passes, update the 360° distance data.

2. Technical Architecture

The program adopts a three-layer architecture of serial reception + ring buffer + protocol parsing, with data passing layer by layer from the hardware to the application.



Data flow: LiDAR UART output → ESP32 UART1 RX (GPIO18) → `Uart1_Rx_Task` reception task → ring buffer → `Lidar_tminiplus_Task` protocol parsing state machine → 360° scan data array.

III. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✔ Required
USB data cable (Type-C)	✔ Required
T-mini Plus LiDAR	✔ Required
Battery (12V)	✔ Required (LiDAR power supply)

2. Prerequisites

- 1. Set up the ESP32-IDF development environment
- 2. Connect the MicroROS-V2 control board to the computer via USB
- 3. Connect the T-mini Plus LiDAR correctly to the control board (UART interface)
- 4. Turn on the chassis power to supply power to the LiDAR

3. Compile and Flash

```
cd ~/esp/base_samples/read_lidar
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

To exit the monitor: press `Ctrl + ]`.

4. Stopping Method

In the terminal, press `Ctrl+]` to exit the serial monitor.

IV. Experiment Result

After flashing and running the program, you can see the following output in the serial monitor terminal:

```
hello yahboom
I (1002) LIDAR_MAIN: Nice to meet you!
I (1012) UART1: Start Uart1_Rx_Task with core:0
I (1012) LIDAR_MAIN: distance:455, 523, 789, 1234
I (1212) LIDAR_MAIN: distance:452, 521, 785, 1230
I (1412) LIDAR_MAIN: distance:458, 525, 790, 1235
```

- A set of distance values (in mm) for 4 angles is output every 200ms
- The 4 values correspond to the 1°, 90°, 180°, and 270° directions
- Cover the LiDAR with a hand or an object in different directions, and the distance value of the corresponding angle will change
- The distance value changes with the actual distance of the obstacle; in an open environment it may show the maximum range value

V. Program Analysis

Source code paths:

- Firmware source: `~/esp/base_samples/read_lidar/main/main.c`
- LiDAR driver:
`~/esp/base_samples/read_lidar/components/lidar_tminiplus/lidar_tminiplus.h`,
`lidar_tminiplus.c`, `tminiplus.h`, `tminiplus.c`
- UART driver: `~/esp/base_samples/read_lidar/components/uart1/uart1.h`, `uart1.c`

1. Directory Structure

```
read_lidar/
├── CMakeLists.txt
├── main/
│   ├── CMakeLists.txt
│   └── main.c
└── components/
    ├── lidar_tminiplus/          # T-mini Plus LiDAR driver encapsulation
    │   ├── lidar_tminiplus.h
    │   ├── lidar_tminiplus.c
    │   ├── tminiplus.h          # Protocol layer
    │   └── tminiplus.c
    ├── uart1/                   # UART1 serial driver
    │   ├── uart1.h
    │   └── uart1.c
    └── ring_buffer/             # Ring buffer
```

```
└─ ring_buffer.h
└─ ring_buffer.c
```

2. Main Program Logic

`main/main.c` implements the LiDAR data collection function:

1. Initialize the UART1 serial port
2. Call `tminiplus_GPIO_Init()` to initialize the LiDAR driver and start the data reception task
3. Send the start command `{0xA5, 0x60}` to wake up the LiDAR and start scanning
4. Loop to read the distance values of 4 specific angles (1°, 90°, 180°, 270°) and print them

```
static const uint8_t s_start_cmd[2] = {0xA5, 0x60};

Uart1_Init();
tminiplus_GPIO_Init();
Uart1_Send_Data((uint8_t *)s_start_cmd, sizeof(s_start_cmd));

while (1) {
    for (int i = 0; i < 4; i++) {
        distances[i] = Lidar_tminiplus_Get_Distance(points[i]);
    }
    ESP_LOGI(TAG, "distance:%d, %d, %d, %d",
              distances[0], distances[1], distances[2], distances[3]);
    vTaskDelay(pdMS_TO_TICKS(200));
}
```

3. Key Components

UART1 serial driver

Files: `components/uart/uart1.h`, `components/uart/uart1.c`

Pin configuration:

Parameter definition file: `~/esp/base_samples/read_lidar/components/uart1/uart1.h`

Signal	GPIO	Description
TXD	GPIO17	UART transmit (connected to the LiDAR RX)
RXD	GPIO18	UART receive (connected to the LiDAR TX)

Communication parameters:

- Baud rate: 230400
- Data bits: 8 bits
- Parity: None
- Stop bits: 1 bit
- Hardware flow control: Disabled

Working mechanism:

`uart1_Init()` initializes the UART hardware and installs the driver, and also creates a reception task `uart1_Rx_Task` that continuously reads data from the UART hardware and stores it in the ring buffer.

Core functions:

- `uart1_Init()` — Initializes the UART1 serial port with a baud rate of 230400 and creates the reception task (stack size 5KB, priority 5)
- `uart1_Send_Data(uint8_t* data, uint16_t len)` — Sends data
- `uart1_Available()` — Returns the number of readable bytes in the ring buffer
- `uart1_Read()` — Reads one byte from the ring buffer

Ring buffer

Files: `components/ring_buffer/ring_buffer.h`, `components/ring_buffer/ring_buffer.c`

Used as a temporary cache for UART data reception, implementing a first-in-first-out data buffering mechanism to avoid data loss.

Buffer size: 1024 bytes (`RX1_BUF_SIZE`).

T-mini Plus LiDAR protocol layer

Files: `components/lidar_tminiplus/tminiplus.h`, `components/lidar_tminiplus/tminiplus.c`

Data structures:

```
#define TMINIPLUS_POINT_NUM 360U // 360 sampling points per revolution

typedef struct __attribute__((packed)) {
    float angle;           // Angle (0°~360°)
    uint16_t distance;     // Distance (mm)
    uint8_t intensity;     // Signal strength
} tminiplus_ros_point_t;

typedef struct __attribute__((packed)) {
    uint16_t size;
    tminiplus_ros_point_t point[360];
} tminiplus_ros_data_t;
```

Data packet format:

A LiDAR data packet consists of a 10-byte fixed header and variable-length sampling data:

Offset	Field	Size	Description
0	PH (packet header)	2 bytes	Fixed at 0x55AA
2	CT (packet type)	1 byte	Packet type
3	LSN (sample count)	1 byte	Number of sampling points in this packet

Offset	Field	Size	Description
4	FSA (start angle)	2 bytes	Angle of the first sampling point
6	LSA (end angle)	2 bytes	Angle of the last sampling point
8	CS (checksum)	2 bytes	XOR checksum
10+	Sampling data	LSN*3 bytes	3 bytes per sampling point (intensity + distance)

Angle calculation formula:

```
// Angle value = (register value >> 1) / 64.0
static float tminiplus_Level1_Start_End_Angle(uint16_t value) {
    return (float)(value >> 1) / 64.0f;
}
```

Distance decoding:

The raw distance value needs to be converted (shift right by 2 bits to remove the interference filtering flag):

```
// Distance value = ((raw value >> 8) << 6) | ((raw value & 0x00FF) >> 2)
static uint16_t tminiplus_Si_To_Distance(uint16_t si_temp) {
    return (uint16_t)(((si_temp >> 8) << 6) | ((si_temp & 0x00FFU) >> 2));
}
```

Data reception state machine:

`tminiplus_Data_Receive()` uses a state machine to parse the serial data stream:

```
RX_WAIT_AA -> RX_WAIT_55 -> RX_CT -> RX_LSN -> RX_FSA_L -> RX_FSA_H
-> RX_LSA_L -> RX_LSA_H -> RX_CS_L -> RX_CS_H -> RX_DATA
```

- After detecting the packet header `0xAA 0x55`, parsing begins
- Calculate the data length based on the LSN field, and receive the complete data packet after confirming it is within the buffer range
- After reception is complete, perform checksum verification
- If verification passes, update the LiDAR data and mark that a new data packet has arrived

Checksum algorithm:

```
result = PH ^ (LSN<<8 | CT) ^ FSA ^ LSA;
for each point: result ^= intensity ^ distance;
// verify that result == CS
```

LiDAR driver encapsulation layer

Files: `components/lidar_tminiplus/lidar_tminiplus.h`,
`components/lidar_tminiplus/lidar_tminiplus.c`

Data reception task:

`tminiplus_GPIO_Init()` creates a dedicated task `Lidar_tminiplus_Task` during initialization (pinned to core 1, stack size 10KB, priority 10). This task continuously reads data from the UART and feeds it into the protocol state machine for parsing. When a complete new data packet is received, it calls `Lidar_tminiplus_Update_Scan_Data()` to update the 360° scan data.

ROS2 message structure initialization:

`Lidar_Ros_Init()` initializes the LaserScan message structure:

Field	Value	Description
angle_min	- π rad (-180°)	Scan start angle
angle_max	+ π rad (+180°)	Scan end angle
angle_increment	$\pi/180$ rad (1°)	Angle increment
range_min	0.12m	Minimum ranging distance
range_max	8.0m	Maximum ranging distance

Data inversion (ENABLE_DATA_INVERSION):

Because the physical installation direction of the LiDAR may be inconsistent with the coordinate system definition of the robot, the `tminiplus_remap_index()` function remaps the angle index to ensure that 0° corresponds to directly in front of the robot. Specifically, the index is shifted by 180° and then modulo is applied.

Core functions:

- `tminiplus_GPIO_Init()` — Initializes the LiDAR driver and starts the data reception task
- `Lidar_tminiplus_Get_Distance(uint16_t angle)` — Gets the distance value at the specified angle (unit mm), angle range 0~359
- `Lidar_tminiplus_stop_Scan(uint8_t on)` — Stops the LiDAR scanning (sends the stop command {0xA5, 0x65})
- `Lidar_tminiplus_Get_Msg()` — Gets the complete LaserScan message (ROS2 message structure)

VI. Common Problems

Problem	Cause	Solution
No LiDAR data output	The LiDAR is not powered or the UART wiring is wrong	Check the battery power supply and confirm that GPIO17/18 are connected correctly

Problem	Cause	Solution
Distance value is always 0	The LiDAR is not started or scanning is stopped	Check whether the start command <code>{0xA5, 0x60}</code> was sent
Data does not update	The serial reception task is blocked or protocol parsing fails	Check the baud rate configuration (should be 230400) and confirm the checksum algorithm is correct
No data at some angles	The corresponding direction is out of range or the LiDAR is blocked	Confirm that the LiDAR rotation has no obstacles and there is a target within 0.12~8.0m
Garbled data	Serial buffer overflow	Increase <code>RX1_BUF_SIZE</code> or raise the priority of the reception task